

Loops

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

The **for** Loop

- Use when you **know how many times** to repeat, or need the index

```
for (let i = 1; i <= 5; i++) {  
  console.log("Count:", i);  
}  
// Count: 1 ... Count: 5
```

- `let i = 1` → start · `i <= 5` → condition · `i++` → update each pass
- All three parts are optional, but you rarely want to omit them

Counting Backwards

```
const items = ["a", "b", "c"];

for (let i = items.length - 1; i >= 0; i--) {
  console.log(items[i]); // c, b, a
}
```

- Reverse order is where an index-based `for` still beats `for...of`
- So is removing items while looping; go backwards so the indexes behind you do not shift

while and do...while

```
let count = 1;
while (count <= 3) {
  console.log("Loop:", count);
  count++; // change the variable or loop forever!
}
```

```
let number = 0;
do {
  console.log("Number is:", number);
  number++;
} while (number < 3);
```

- `do...while` always runs the body **at least once**

for...of and for...in

```
const fruits = ["apple", "banana", "orange"];
for (const fruit of fruits) {
  console.log(fruit); // each value, no indexes
}
```

```
const user = { name: "Sam", age: 25 };
for (const key in user) {
  console.log(key, "=", user[key]); // keys of an object
}
```

- `for...of` for array **values**; `for...in` for object **keys**
- Do not use `for...in` on an array; you get index *strings*

break and continue

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue; // skip the rest of this pass  
  if (i === 5) break;    // stop the loop entirely  
  console.log(i);  
}  
// 1  
// 2  
// 4
```

- `continue` skips one pass; `break` leaves the loop for good

Choosing a Loop

Need	Reach for
Every value in an array	<code>for...of</code>
The index too	<code>for</code>
Reverse order	<code>for</code> counting down
Unknown number of passes	<code>while</code>
Always run at least once	<code>do...while</code>
Every key of an object	<code>Object.entries</code> + <code>for...of</code>

- Transforming or filtering? Use `map` / `filter` instead; next pair